1. Koje tri komponente se smatraju ključnim za uspeh DevOps koncepta.
Ljudi, procesi, tehnologije

2. Da li se u DevOps pristupu potencira distinkcija između timova koji rade razvoj i timova koji operativno upravljaju softverom, ili se teži da barijera između ovih timova iščezne?

U DevOps-u, izolovanje razvojnih i operativnih timova nestaje. Ali cilj nije samo da ovi timovi počnu blisko da sarađuju, već da se uspostavi takvo okruženje u kojem članovi razvojnog tima u svakoj fazi razvoja proizvoda (softvera) rade zajedno sa operativnim timovima (koji imaju najveći uvid u očekivanja krajnjih korisnika i kako se aplikacija zaista koristi). Drugim rečima, sama distinkcija između ovih timova bi trebala da iščezne i da se oni uvek posmatraju kao jedan integralni DevOps tim.

3. DevOps procesi teže da maksimalno koriste koji princip?
Kako bi se obezbedilo da ponavljajući manuelni zadaci ne usporavaju razvoj i isporuku proizvoda, DevOps procesi teže da se baziraju u maksimalno mogućoj meri na konceptima “infrastructure as code” što omogućava timovima da automatizuju mnoge korake u životnom ciklusu razvoja i produkcione upotrebe softverskih proizvoda - obezbeđivanje i upravljanje neophodnim infrastrukturnim resursima. Primer: (instalacija i konfigurisnje servera).

4. Koje tehnologije obezbeđuju dovoljno brzu i fleksibilnu platformu za DevOps?
Osnovni pokretač uspešne primene DevOps principa je upotrebna alata koji omogućavaju automatizaciju i upravljanje određenim fazama u životnom ciklusu razvoja proizvoda (softvera) • Zrelost tehnologija virtuelizacije, računarstva u oblaku i pojava IaaS su obezbedili dovoljno fleksibilnu, efikasnu i brzu platformu koju DevOps zahteva. Danas se uobičajeno spominju Docker, Kubernetes, GitHub, Jenkins, Travis CI … alati koji se koriste u DevOps. Ali da li su oni suština DevOps? - NE, oni bi se podjednako dobro mogli koristiti i u tradicionalnom pristupu, ključno je kako se neki alat koristi. Ne postoji skup predefinisanih alata koji su “DevOps alati” već postoje različiti alati za različite namene koji mogu biti korišćeni u kontekstu DevOps. - Istovremeno i kontejnerizacija i kontrola izvornog koda i kontinualna integracija (sve ono za šta se koriste gore pomenuti alati) JESU operacije koje imaju veliku ulogu u DevOps kontekstu.

5. Koji su problemi često pratili razvoj softvera i doveli do stvaranja koncepta DevOps-a?
Razvojni timovi i sistem administratori dosta često mogu imati neusaglašeno dejstvo, primarno uzrokovane čestim novim zahtevima krajnjih korisnika - koji žele nove servise, nove funkcionalnosti što je brže moguće. Ali istovremeno klijenti očekuju stabilno okruženje (bez otkaza i prekida u isporuci servisa). • Ovo je često dovodilo do krucijalne dileme - obezbediti brze izmene (što razvojni timovi mogu da obezbede) ne vodeći mnogo računa o uticaju promena na stabilnost, ili usporiti razvoj kako bi se potvrdila stabilnost sistema. • Nijedna od ovih krajnosti na kraju nije zadovoljavajuća za krajnjeg klijenta.
DevOps je osmišljen upravo sa ciljem da objedini sve koji su povezani sa razvojem softvera i njegovim puštanjem, održavanjem i korišćenjem u produkcionom okruženju - poslovne korisnike, razvojne timove, test inženjere, sistem administratore

6. Koji ključni koncepti čine suštinu funkcionisanja DevOps-a?
Kako bi se obezbedilo da ponavljajući manuelni zadaci ne usporavaju razvoj i isporuku proizvoda, DevOps procesi teže da se baziraju u maksimalno mogućoj meri na konceptima

“infrastructure as code” što omogućava timovima da automatizuju mnoge korake u životnom ciklusu razvoja i produkcione upotrebe softverskih proizvoda - obezbeđivanje i upravljanje neophodnim infrastrukturnim resursima. Primer: (instalacija i konfigurisnje servera)

7. Koja su tri osnovna principa DevOps-a?

• Princip toka (engl. Flow) - ubrazava isporuku rezultata rada (razvoja) od razvojnog tima ka operativnom timu i klijentu • Princip povratnih informacija (engl. Feedback) - ustanovljavanje ovog principa omogućava povećanje bezbednosti i kvaliteta rada • Princip kontinualnog učenja i eksperimentisanja - promoviše kulturu visokog poverenja i naučnih principa pri preuzimanju rizika u svakodnevnim poslovima

8. Šta je value stream / techonology value stream?
“Sled aktivnosti koju određena organizacija sprovodi kako bi ispunila zahteve korisnika”. • “Sled aktivnosti koje su neophodne da se izvrši dizajn, proizvodnja i isporuka dobara i usluga krajnjem korisniku, ubuhvatajući pri tome dvosmerni tok informacija i materijala”. Value Stream Mapping: How to Visualize Work and Align Leadership for Organizational Transformation Karen Martin & Mike Osterling • U proizvodnim delatnostima ovaj tok stvaranja nove vrednosti je lako identifikovati - započinje narudžbom kupca i isporukom materijala od koga počinje proizvodnja

9. Koje mere pokazuju efikasnost organizacije u izvršavanju poslovnog procesa?
Vreme isporuke - Deployment Lead Time • Procesno vreme - Process Time • %C / A mera

10. Šta je vreme isporuke - Deployment Lead Time?
Vreme isporuke - vremenski interval koji protekne od momenta kada neki zahtev bude prihvaćen za realizaciju do momenta kada je on uspešno primenjen u produkciji. • Skraćenje ovog vremena je u fokusu većine praksi DevOps-a- Početna faza - Dizajn i razvoj (engl. Design and Development) zahteva visok nivo kreativnosti i dosta uloženog, često jedinstveno utrošenog rada (koji neće biti ponavljan). Ovde je utrošeno vreme često vrlo varijabilno.- Druga faza - Testiranje i operativni nadzor (engl. Testing and Operations) - takođe zahteva kreativnost i ekspertizu, ali je predvidljivog vremena trajanja, a sam cilj je da se postignu zadovoljavajući rezultati sa minimizacijom varijabilnosti

11. Šta je procesno vreme?
Procesno vreme je vremenski interval koji protekne od momenta kada se započne razvoj nekog prihvaćenog zahteva do momenta kada je on uspešno u produkciji

12. Šta prikazuje mera procentualnog odnosa C/A?
Procenat kompletno i tačno realizovanih zahteva je treća mera koja ukazuje na efikasnost organizacije • Može se utvrditi tako što se anketiraju krajnji korisnici koji procenat onoga što im je isporučeno mogu odmah, bez ikakvih izmena i prilagođavanja da koriste u datom stanju - bez potrebe da se ponovo pojašnjava zahtev koji su imali

3 13. Koje prakse čine suštinu uvođenja „principa toka“ i omogućavaju optimizaciju vremena isporuke?

učiniti rad vidljivim • ograničiti broj aktivnih zadataka • smanjivanjem veličine i vremena potrebnog za jedinicu rada (small batch sizes) • smanjenje broja primopredaja (prosleđivanja) zadataka • kvalitet se integriše na samom početku - time se smanjuje propagacija grešaka

14. Šta se postiže praksom koja omogućava da je svaki rad vidljiv?

Ključna razlika između klasičnih proizvodnih procesa i visokotehnoloških procesa je u tome što se u klasičnim procesima uvek vidi do koje faze je proizvodnja stigle, i lako je otkriti tačke u kojima dolazi do kašnjenja nagomilavanja… • U tehnološkom toku kreiranja nove vrednosti, ovo prosto nije golim okom vidljivo, radni zadaci se mogu preneti na sledeći tim klikom dugmeta, ali isto tako može doći i do spolja neuočljivog zastoja ili beskonačnog "prebacivanja" zadataka zbog nekompletnosti informacija. • Kako bi se poboljšala vidljivost onoga što se radi neophodno je koristiti neke pomoćne alate - table sa radnim zadacima (kanban) ili neki drug vid rešenja gde se pojedinačni zadaci mogu vizuelizovati u vidu kartica i lako identifikovati gde se nalaze u procesu rada. Pomoću kanban tabli radni zadaci ne samo da postaju vidljivi, već se jasno vidi i njihov napredak (levo na desno) kroz kolone koje predstavljaju stanje samo zadatka, • Može se meriti i ukupno vreme isporuke - do momenta kada je zadatak smešten u kolonu "završeno". Dobra organizacija podrazumeva da se zadatak smešta u "done" samo kada je stvarno implementiran u produkciji - isporučen klijentu

15. Koje tehnike možemo koristiti da učinimo rad vidljivijim?
Pogledati prethodni odgovor

16. Zašto je bitno ograničiti broj aktivnih zadataka?
Limit Work in Progress • U klasičnoj proizvodnji dnevni zadaci su tipično unapred jasni i određeni, a njihovo prekidanje je odmah uočljivo • U visokotehnološkim procesima, dnevni radni zadaci su mnogo dinamičniji, vrlo često se prioriteti određuju na dnevnoj bazi, pri čemu hitni zahtevi pristižu sa raznih strana (ticketing, hitni pozivi zbog otkaza nekog sistema, emailovi). • Prekidanje tekućih radnih zadataka u IT sferi je istovremeno i manje vidljivo, ali potencijalno "preotimanje" radnog vremena pojedinaca može drastično uticati na produktivnost celog tima
Na primer razvojni inženjer koji radi na više projekata pri promeni zadatka na kome radi praktično gubi vreme jer mora se mora prebaciti na drugo okruženje, prebaciti i svoje razmišljanje u novi kontekst • Vreme potrebno da se pojedinačni zadatak završi drastično se povećava ako zahtevate od osobe da se stalno prebacuje i paralelno rešava nekoliko problema (multitasking) • Multitasking se može ograničiti upotrebom kanban table - tako što se usvoji pravilo koliko maksimalno Work in progress kartica može biti u svakoj koloni. • Istovremeno ovakvo limitiranje čini vidljivijim i one zadatke koji iz nekog razloga ne mogu da se kompletiraju

17. Zašto je pogodno smanjiti veličinu i vreme isporuke pojedinačnih zadataka?
Da bi proces rada bio što protočniji jedan od ključnih principa je "razbijanje" posla na male jedinice rada (male blokove) • Veliki blokovi radnih zadataka neminovno povećavaju broj Work in progress stavki jer ih je nemoguće savladati u kratkom vremenu • Mali blokovi radnih zadataka omogućavaju da se kompletan posao na određenom zadataku završi - i time isporuči klijentu, u protivnom on ima jako dug period u kome čeka da dobije bilo koju

novu funkcionalnost • Dodatno isporuka velikih blokova dodatno komplikuje deployment jer je mnogo veća verovatnoća da se tada jave problemi za korisnika • Kad god postoji jako dugo vreme isporuke tipično je izazvano potrebom da se obavi jako veliki broj operacija kako bi se kod prebacio iz jednog u drugo okruženje. Prosleđivanje koda iz jednog u drugo okruženje zahteva učestvovanje različitih strana i te primopredaje usporavaju proces. •

18. Zašto je pogodno smanjiti broj „primopredaja“ jedinica rada u procesu rada?
Kada god se kod prosleđuje između timova to podrazumeva dosta komunikacije - zahteve, potvrde, pojašenjenja, koordinasinje, planiranje, uklanjanje konflikta u kodu. • I pri najboljoj komunikaciji po pravilu se određene informacije “zagube” u ovim primopredajama. • Ključna stvar je stoga redukovati broj ovakvih primopredaja - poželjno tako što se operacije automatizuju ili tako što se timovi reorganizuju tako da se više koraka obavlja unutar jednog integrisanog tima

19. Zašto je važno smanjiti količinu „otpada“ koja se uklanja iz procesa rada?
Otpad - dakle neiskorišćeni, a uloženi rad je najveća opasnost za bilo koji produkcioni sistem. • Upotreba bilo kog resursa ili materijala preko mere koju očekuje krajnji klijent i što je spreman da plati je neisplativa. • Sam koncept eliminisanja “otpada” se smatra dehumanizovanim i sa psihološkog stanovišta demotivišućim, pa se umesto toga napori usmeravaju ka tome da se iz procesa ukloni bespotreban težak i opterećujući rad, čiji rezultati nisu u direktnoj korelaciji sa krajnjim ciljem za klijent

20. Šta obezbeđuje princip povratnih informacija?
opisuje principe i tehnike koji su potrebni kako bi se obezbedio brz i konstantan protok povratnih informacija (od operativnog tima ka razvojnom) - tok sdesna ulevo u svim razvojnim fazama
Ovo je posebno važno kada se radi o kompleksnim sistemima, kod kojih bi, bez uvođenja ovakvih principa, prva prilika da se uoči i ispravi greška bila tek kada je neki katastrofalni otkaz već u toku

21. Problemi u kompleksnim sistemima pre ili kasnije moraju da se dese - šta sigurno radno okruženje predviđa kao prihvatljiv mehanizam rešavanja problema?
Ovo omogućava da se problemi otkriju u ranim fazama, dok su još mali, jeftiniji i jednostavniji za ispravku, i sprečimo njihovo ispoljavanje u katastofalnom obliku
…
Složenim radnim zadacima se upravlja na takav način da se pravovremeno otkriju problemi u dizajnu i razvoju proizvoda • Problemi se grupišu i rešavaju, što dovodi do brzog usvajanja novog znanja • Ovako nastalo novo (lokalno) znanje se brzo širi i koristi u svim delovima organizacije • Vođe konstantno stvaraju nove vođe koji kontinuirano napreduju u određenim oblastima
…
Ključni mehanizam za ovo je uspostavljanje povratnih petlji, ali i prosleđivanja informacija unapred u razvojnom okruženju.

22. Povratni tokovi omogućavaju pravovremeno otkrivanje problema, zašto je to bitno?
Pogledati prethodni odgovor.
Povratni tok informacija ne samo do omogućava brzo otkrivanje i oporavak od grešaka, već obezbeđuju i vredne informacije kako izbeći da se problem ponovi.

• Svaki član tima treba da se brine za kontrolu kvaliteta i ispravljanje grešaka u delu nad kojim ima uvid i kontrolu. Na ovaj način se kontrola kvaliteta i donošenje o tome približava mestu gde se stvarni rad i obavio.
• Koristi se koncept odobravanja od strane timskih kolega kako bi se potvrdilo da unesene i predložene promene rade kako je predviđeno (i ne narušavaju prethod funkcionalnosti)
• Automatizuje se koliko je god moguće provera kvaliteta koju obavljaju QA i Info security. Na taj način umesto da razvojni tim mora da "traži" da se organizuju tes automatizovani testovi se mogu izvršavati "po potrebi", tako da razvojni tim može brzo i vrlo često da testira novi kod.
• Na ovaj način kvalitet postaje nešto o čemu se brinu svi tokom svake faze razvoja, a ne zadatak nekog izolovanog, nedostupnog tima.

23. Šta bi bili pravilni principi koji promovišu kvalitet na samom izvoru?

24. Koje su osnovne ideje principa konstantnog učenja i eksperimentisanja?
• Promoviše se kultura rada koja omogućava konstantno učenje i ekperimentisanje. • Na taj način omogućava se kreiranje konstantno novog individualnog znanja, koje se zatim prenosi na cele timove i organizaciju. • Kod proizvodnih sistema, pa tako i razvojnih timova za razvoj softvera, problemi vrlo često proističu iz činjenice da su previše rigidno postavljeni, tako da zaposleni (u bilo kojoj roli) imaju vrlo malo mogućnosti da poboljšanja predlože i ugrade u svakoddnevnu radnu rutinu. • Vrlo često je ovo na kraju kombinovano sa celom kulturom ponašanja na radnom mestu u kojoj dominira strah od nadređenih i malo poverenja u radne kolege, i svi koji ukazuju na probleme se vrlo često gledaju kao neko ko stvara probleme (reklo bi se "talasa"). • Nasuprot tome, visokoproduktivne organizacije aktivno promovišu učenje novih stvari (čak ponekad i zahtevaju), omogućavajući da radne aktivnosti vremenom evoluiraju usvajanjem novih, boljih tehnologija i tehnika rada. • Zaposleni često imaju priliku da eksperimentišu sa novim rešenjima kako bi se uverili da su podobna za određenu upotrebu. • U visokotehnološkim razvojnim procesima cilj je stvoriti atmosferu poverenja koja stimuliše konstatno učenje, i ljude koji su spremni proceniti prihvatljive rizike i prihvatiti eksperimentisanje kao način da se dođe do naprednih rešenja.

25. Da li se uči samo iz uspešnih pokušaja ili neuspešni pokušaji implementacije određenih rešenja takođe povećavaju nivo znanja?
Uči se i iz uspešnih i iz neuspešnih pokušaja. One ideje koje "ne rade" se identifikuju i svima se pojašnjava zašto takvo rešenje nije dobro za određeni problem, a ona koja "rade" se potvrđuju i preporučuju svima. • Stalno se unosi određena doza "stresa" u celokupni sistem koja promoviše kontinualno poboljšavanje rešenja.

26. Jedan od načina reagovanja upravljačkog tima na uočene greške je i "name, blame and shame". Da li takav pristup trajno podstiče razvoj organizacije?
Odgovor na incidente koji se od strane zaposlenih doživljava kao nepravedan, može zapravo onemogućiti dobru analizu sigurnosnih propusta, promovisati atmosferu straha umesto atmosfere posvećenosti kod onih koje obavljaju kritične operacije, i birokratizovati organizacije umesto da ih učini pažljivijim, i da rezultuje u razvoju radne kulture u kojoj postoje profesionalne tajne, izbegavanje i zaštita ličnog interesa

27. Organizacije koje se po načinu funkcionisanja mogu svrstati u "patološke" karakteriše kakvo radno okruženje?
Patološke - karakteriše ih atmosfera straha i pretnji - informacije se vrlo često skrivaju

28. Šta bi bile "generativne" organizacije?
Generativne - u ovim organizacijama se aktivno traži i podstiče deljenje informacija kako bi se poboljšali krajnji rezultati.

29. Zašto je bitno sprovesti proces pretvaranja lokalnih saznanja u globalna poboljšanja na nivou organizacije.
Kada se u jednom segmentu dođe do novih saznanja u organizaciji mora postojati razrađen mehanizam da se ta saznanja prošire do svih kojima su potrebna. • Implicitno saznanje do kojih je došao jedan tim, a poželjno je i za ostale, treba prevesti u dokumentovano znanje

dostupno svima • Na taj način svi koji se mogu susresti sa sličnim problemom mogu već unapred znati dobar način da ga razreše

30. Šta omogućava institucionalizacija unapređenja dnevnih aktivnosti? uvodjenje otpornosti koda - game day
Rezultat je da svako pronalazi i uklanja probleme uočene u onom delu koda za koji je nadležan, i to konstantno, kao deo svakodnevnih aktivnosti. Kada se sistematskim pristupom uklone dugotrajno postojeći problemi, onda se može stići i do tačke da ima vremena za uklanjanje i jednostavnijih ili manje vidljivih problema.
Sve ovo na kraju dovodi do toga da imamo organizaciju koja probleme uklanja odmah kada se uoče, kada je to mnogo jefitnije i manje vremenski zahtevno.

31. Zašto je bitno da deployment pipeline bude tako organizovan da obezbeđuje deployment na okruženje što sličnije krajnjem produkcionom?
Vrlo često se dešava da razvojni timovi nemaju nikakav osećaj niti informaciju kako se aplikacija ponaša u stvarnim, produkcionim okruženjima. • Prve informacije i iskustvo dobiju tek kada se aplikacija stvarno instalira na produkcioni sistem, a to je prekasno i potencijalno opasno po poslovne potrebe klijenta

32. Zašto je bitno da se okruženja kreiraju automatizovano I po mogućnosti “na zahtev”?
• Kako bi se obezbedilo uspostavljanje brzog i pouzdanog toka (informacija i urađenog posla) od razvojnih ka operativnim timovima neophodno je obezbediti da se u svim fazama u razvojnom lancu koriste okruženja koja su ekvivalentna produkcionom okruženju. • Potrebno je da se ova okruženja mogu kreirati automatizovano, korišćenjem skriptova i konfiguracionih informacija koje su smeštene i čuvaju se u nekom od sistema za kontrolu verzija.
Idealno - okruženja se na osnovu ovih skriptova mogu kreirati “na zahtev”, kada su potrebna, samostalno, bez potrebe za kreiranjem “tiketa” i bez potrebe da se nešto ručno podešava.

33. Navesti neke od operacija koje je moguće automatizovati, a koje učestvuju u procesu formiranja odgovarajućih okruženja.
automatizovane skriptove za kreiranje kontrolisanih i pouzdanih okruženja?

34. Zašto je bitno da deployment pipeline počiva na upotrebi repozitorijuma za kontrolu verzija?
Timovi su onda pažljivo odradili prenos svih produkcionih izmena u sistem za kontrolu verzija i napravili automatizovane skriptove za kreiranje kontrolisanih i pouzdanih okruženja • Vreme potrebno da se dobije odgovarajuće okruženje palo je sa 8 nedelja na 1 dan • Upravo ova promena omogućila je da timovi konačno počnu da postižu zadane rokove, uklope se u predviđene troškove i što je jako bitno smanje broj grešaka koje su se “probile” u produkciona okruženja klijenta

35. Šta bi sve trebalo da se nalazi na sistemu za kontrolu verzija?
Kako bi se garantovalo da i posle eventualnog katastrofalnog otkaza možemo sisteme oporaviti u kratkom vremenu, na sistemima za kontrolu verzija bi trebalo da se nalaze: • sav aplikativni kod i dodatni kod od kojeg je on zavisan • svi skriptovi za kreiranje baza podataka, statičkih i sl. • svi artefakti neophodni za kreiranje okruženja (VMWare ili AMI slike, Puppet ili slični skriptovi) • svi fajlovi za kreiranje kontejnera (Docker slike ili compose fajlovi) • svi

automatizovani testovi (i opisi manuelnih testova) • svi skriptovi koji obezbeđuju pakovanje koda, instalacione skripte, skripte za migraciju baza podataka… • drugi projektni artifakti (dokumentacija korisničkih zahteva, procedure za instaliranje…) • svi konfiguracioni fajlovi za instaliranje na cloud platformama (AWS CloudFormation templates, Azure Stack DSCs…)

36. Zašto je bitno napraviti okruženje u kome je lakše i jeftinije rebildovati infrastrukturu nego je popravljati?
Skratiti ovo…

Ukoliko je kompletno obezbeđivanje infrastukture zasnovano na automatizovanim skriptama, moguće je doći u stanje kada je lakše i jeftinije iznova pokrenuti nove instance neophodne infrastrukture nego trošiti vreme na oporavljanje konfiguracija koje su otkazale

Kada postoje automatizovane (ponovljive) skripte koje obezbeđuju potpuno konfigurisanu i funkcionalnu infrastrukturu, moguće je obezbediti i dodatne kapacitete kada se za to ukaže potreba (horizontano skaliranje)

• Ovakvim pristupom se izbegavaju i katastrofalne posledice otkaza infrastrukturnih komponenti koje bi bilo nemoguće iznova reprodukovati jer su u njih uloženi sati manuelnog konfigurisanja

Kako bi se obezbedila konzistentnost infrastrukturnih komponenti kada god se napravi podešavanje konfiguracije, kreira određeni patch, upgrade na novu verziju, sve ove promene treba konzistentno ponoviti u svim preprodukcionim i produkcionim okruženjima.

• Mnogo je lakše ukoliko se usvoji praksa da se nijedna konfiguracija ne unosi ručno u pojedinačne sisteme, već da se isključivo ažuriraju konfiguracione skripte koje će datu izmenu dosledno primeniti na svim okruženjima.

• Dva pristupa - korišćenje alata koji automatizuju ove poslove i garantuju konzistenciju (Ansible, Puppet, Chef…), ili pristup kod koga svaka izmena kreira novu verziju fajlova za VM ili slične infrastrukturne komponente

• Ovaj drugi pristup predstavlja immutable infrastructure pattern • nikakve izmene na “živim” instancama infrastrukturnih komponenti se ne dozvoljavaju • svaka promena se prvo unosi u automatizovane skripte za bildovanje i deployment • kreiraju se nove instance koje preuzimaju zadatke postojećih, a stare (postojeće) se potom gase

• Ponekad se prihvata i praksa da se infrastrukturne komponente namerno gase i zanavljaju novim automatsi kreiranim iz skripti, baš sa ciljem da se kod svih učesnika u lancu razvoja razvije svest o tome da se ne mogu osloniti ni na kakvo specifično podešavanje sistema - sve mora raditi na komponentama koje se kreiraju iz određene verzije skriptova za dato okruženje

37. Zašto je pogodno insistirati na kratkim ciklusima razvoja?

38. Koji problemi nastaju ako se testiranje odlaže I tretira kao poseban deo razvojnog ciklusa koji se sprovodi povremeno?

Ukoliko se velika testiranja obavljaju nekoliko puta godišnje u velikim “paketima” testova, programeri o potencijalnim greškama saznaju prekasno, pri čemu je i sećanje o unetim promenama koje mogu izazvati uočene greške već izbledelo (što naravno otežava uklanjanje grešaka)

Dodatni problem - što više koda se isporučuje, to više vremena i novca treba uložiti u testiranja - proizvodnja kompleksnih softverskih sistema postaje problematična jer postaje neskalabilna

- Automatizacija testiranja koje se obavlja nakon svakog commit-a je rešenje za ovakve problem

39. Koje su ključne ideje kontinualnog testiranja?
Cilj je da se o kvalitetu proizvoda vodi računa od samog početka
• Svaki programer kreira automatizovane testove kao deo svog posla •
Na ovaj način obezbeđuje se brza povratna veza - ako nešto ne radi, to se vidi skoro odmah po komitovanju

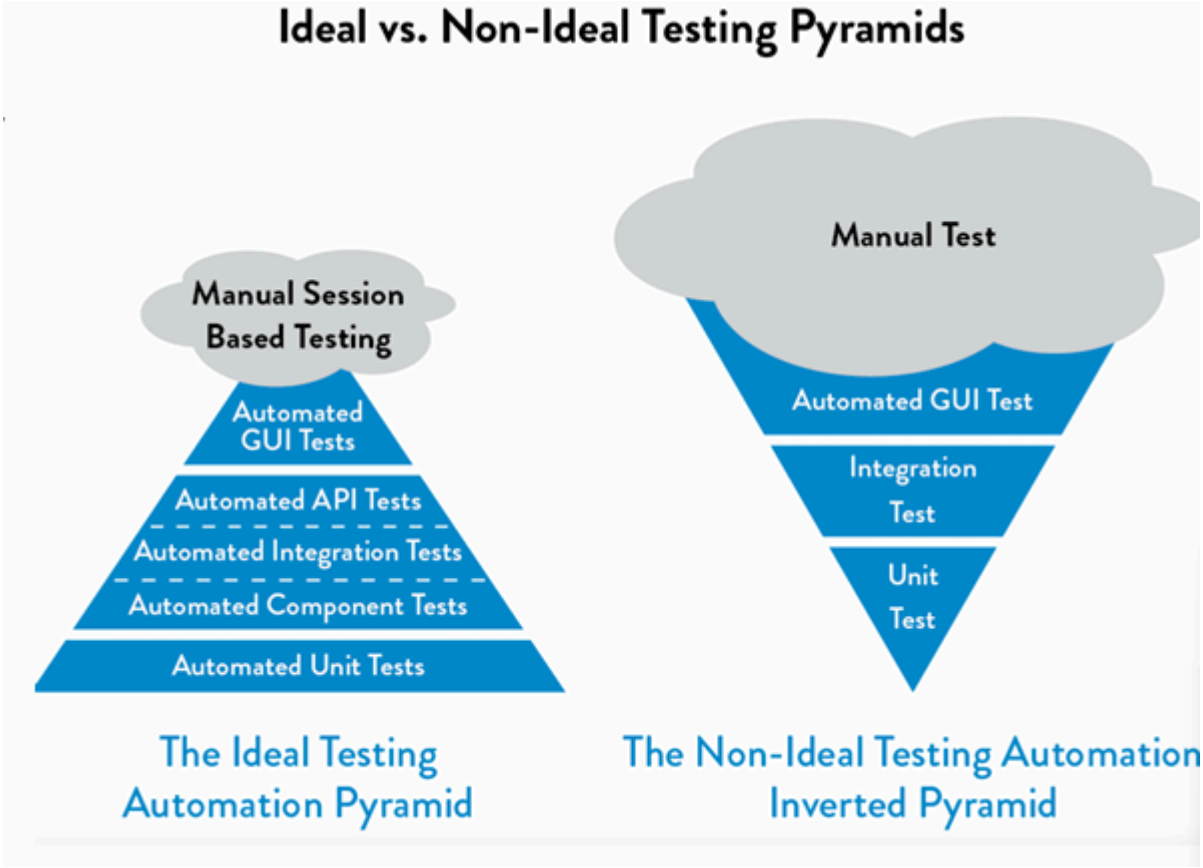
40. Kakva je uloga automatizovanog testiranja u kontinualnom testiranju?
Automatizovani test paketi (test suites) povećavaju frekvenciju integracije i testiranja koda, donoseći suštinsku promenu iz povremenog u kontinuirano.
• Ovo se postiže tako što se razvija deployment pipeline čija primaran uloga je da pokreće integraciju i automatsko testiranje čim se nova promena koda unese u sistem (komituje na sistem za kontrolu verzija)- bildovanje i testiranje se automatski obavlja na okruženjima koja odgovaraju produkcionim

41. Kako izgleda proces funkcionisanja deployment pipeline-a?
Proces rada pipeline-a:- pokreće se kada se izvrši commit - nakon čega se izvršava automatsko bildovanje i integrisanje komponenti- pokreću se automatizovani Unit testovi ‣ mogući su i dodatni koraci provere (statička kod analiza, provera dupliranog koda, provera pokrivenosti koda testovima)- nakon svih uspešnih testova pokreće faza prihvatanja (acceptance) ‣ automatski se deployuju svi prethodno kreirani paketi na production like okruženje i pokreću skupovi automatizovanih acceptance testova- nakon što prođu acceptance testovi, kod se prihvata na sistemu za verzioniranje i pokreće se pakovanje distributivne verzije (čime se garantuje da se ista distibucija koristi svuda, i uklanja problem različitih varijanti iste verzije softvera)

42. Koje su pretpostavke za uspostavljanje prakse kontinualne integracije?
Nakon što je uspostavljen deployment pipeline, neophodno je usvojiti praksu kontinuirane integracije za šta postoje tri pretpostavke:
- postojanje sveobuhvatnog skupa automatizovanih testova čijim uspešnim izvršavanjem se garantuje da je kod u isporučivom (deployable) stanju
- kultura rada koja podrazumeva da se obustavi rad kada se uoči da su validacioni testovi “pali”
- programeri moraju razvijati kod u malim “paketima” koji se onda spajaju sa glavnom granom dosta često, a ne da razvijaju kod u dugotrajnim granam u kojima se razviaju komplikovane promene

43. Šta je smisao rane detekcije grešaka automatizovanim testovima?
Smisao automatizovanih testova je da nam omoguće što raniju detekciju grešaka • Brzi (unit testovi) rade se pre složenijih (acceptance testova), a oni opet pre bilo kakvog manuelnog testiranja • Jedno od željenih stanja je da se svaka greška potencijalno otkrije u bržim fazama testiranja. Ako se greške otkrivaju tek u kasnijim fazama, brzina protoka povratnih informacija ka programerima jako pada, a reprodukovanje same greške je teže • Kada god se neka greška otkrije u acceptance ili integration fazi testiranja, neophodno je razmisliti koji unit testovi se mogu napisati kako bi se takva greška otkrila ranije

44. Šta je idealna test piramida?



45. Šta je koncept Test Driven Developmenta?

Tri koraka ‣ inicijalno stanje - napisati test koji “ne prolazi” (jer nema koda koji bi uspešno izvršio test). Komitovati kod. ‣ programirati funkcionalni kod dok ne zadovolji test. Komitovati kod. ‣ refakturisati i stari i novi kod kako bi bio dobro strukturisan, pazeći pri tome da testovi i dalje “prolaze”. Ponovo komitovati kod

46. Šta je koncept kontinualne integracije?

kontinualna integracija - promena filozofije - neka spajanje koda na glavnu granu (naravno uz sve prethodne korake koji garantuju da taj kod ne “ruši” pipeline) treba da postane svakodnevna aktivnost svakog programera

47. Šta su i koje prakse minimizuju integracione probleme?

Razvoj koda u malim “paketima” i izbegavanje razvoja u dugo izdvojenim granama sa retkim spajanjima- Razvoj velikih feature-a je kontraproduktivan jer dugo traje, dugo se zadržava kod u izolovanim granama, i na kraju se javlja veliki broj integracionih problema. Mali taskovi se obično realizuju brzo, sa malim izmenama koda koje se brzo spajaju (sa vrlo malo problema). - Strategije grananja se mogu svrstati u dve glavne grupe: optimizovanje produktivnosti pojedinca (zasniva se na privatnim granama), i optimizovanje produktivnosti tima (zasniva se na radu na zajedničkoj grani)

Usvajanje koncepta razvoja na glavnoj grani- Kontramera za sprečavanje predugog rada na preglomaznim izmenama koda.- Od programera se traži da bar jednom dnevnom svoj kod spoje na glavnu granu. Ovo nas tera da razvoj razlomimo u manje segmente, tako da imamo šta da komitujemo. Ukoliko spajanje rade češće - to bolje.- Ovo omogućava da se često

“vrte” testovi koji brzo otkrivaju kod koji unosi grešku u pipeline i da se on odmah koriguje. ‣ gated commit - konfigurisanje pipeline-a tako da se odbija sav kod koji pipeline izvodi iz deployable stanja. U ovom slučaju prvo se proverava da li je kod moguće spojiti, da li se bild izvršava normalno i prolaze svi automatizovani testovi i teko potom se spajanje zaista obavlja na samu glavnu granu

48. Zašto je potrebna automatizacija procesa isporuke?
• Ispostavilo se da je za veći broj funkcionalnih izmena neophodno veći broj deployment-a, jer se pokazuje da postoji limit broja izmena koje uspeju da se uključe u jedan deployment.
• Ali kada se koristi kontinualna integracija i kada je sav kod već proveren onda je moguće obezbediti i automatizaciju samog procesa deploymenta, što čini sam postupak predvidljivim i niskorizičnim.
• Ovo značajno podiže produktivnost, jer proces manuelnog depolymenta je vremenski zahtevan, sklon greškama, i vrlo često je podrazumevao proces primopredaje između razvojnog i operativnog tima.
• Automatizacija procesa deployment-a obezbeđuje mogućnost promocije proverenog koda u produkciono okruženje (na zahtev ili automatizovano - odmah posle uspešnog builda i uspešno prođenih svih testova)

49. Koje korake procesa isporuke se može automatizovati?
• Neophodno je dokumentovati eventualno postojeći proces isporuke, a zatim sageldati koje je sve korake moguće automatizovati: • pakovanje koda • kreiranje unapred konfigurisanih slika virutuelnih mašina ili kontejnera • automatizacija deploymenta i konfiguracije middlewear-a • kopiranje paketa (ili fajlova) na produkciono okruženje • restartovanje servera, aplikacija ili servisa • generisanje konfiguracionih fajlova iz postojećih template-a • pokretanje smoke testova kako bi se uverili da sistem posle instalacije radi i da je korektno konfigurisan • izvršavanje test procedura • skripte i automatizacija neophodnih izmena u bazama podataka

50. Zašto je bitno da razvojni timovi učestvuju u deployment-u?
razvojni tim treba da je uključen u automatizaciji i optimizaciji procesa isporuke, kako bi stekao iskustvo i postali svesni zavisnosti - ovo sikustvo nakandno im pomaže da se izbegnu situacije da male izmene u aplikaciji zahtevaju ponovni deployment ili promenu okruženja

51. Koje zahteve treba da ispune alati koji potpomažu automatizaciju isporuke?
Istovetna isporuka na sva okruženja • korišćenje istog mehanizma isporuke za različita orkuženja (development, test, production) smanjuje mogućnost problema na produkciji • Smoke testiranje isporučenog koda • Obezbediti da se koriste konzistentna okruženja • Kada je uspostavljena istovetnost bilda za sva okruženja, neophodno je obezbediti da ta okruženja ostanu međusobno sinhronizvoana ako dolazi do nekih promena

52. Koje prednosti donosi mogućnost isporuke koda „na zahtev“?
brza reakcija na potrebe korisnika

53. Zašto je dobro razdvojiti isporuku (deployment) i lansiranje nove verzije (za korisnike aplikacije)?
Striktno razdvajanje isporuke koda (deployment) i lansiranja verzije za klijente pomaže da se prevaziđu ovi problemi • Isporuka koda (deployment) predstavlja instalaciju specifične

verzije softvera na određeno okruženje. Sama isporuka ne mora biti istovremeno povezana i sa objavljivanjem određenih funkcionalnosti krajnjim korisnicima. • lansiranje verzije koda se dešava kada se određene funkcionalnosti ili grupa njih učine dostupne krajnjim klijentima. Kod i sama arhitektura sistema treba da su takvi da omogućavaju da se ovo lansiranje novih funkcionalnosti obavlja bez potrebe za dodatnim izmenama u kodu. • Ukoliko su proces instalacije i lansiranja određenih funkcionalnosti odvojeni, onda objavljivanje novih funkcionalnosti više nije tehnički problem (jer sve je većć ranije instalirano i testirano), već pitanje poslovne odluke kada omogućiti pristup određenim funkcionalnostima

54. Koji obrasci za lansiranje novih verzija aplikacija postoje?
Postoje dve kategorije šablona za lansiranje novih verzija koda: • lansiranje (release) na osnovu okruženja - kada imamo dva ili više okruženja na koje se instalira kod, ali samo jedno okruženje prima živi saobraćaj od krajnjih korisnika. Novi kod se instalira na ona okruženja koja ne primaju živi saobraćaj i tamo se testira. U tom slučaju lansiranje nove verzije se izvodi preusmeravanjem saobraćaja na drugo okruženje. - blue-green deployment, canary releases, cluster immune systems • lansiranje (release) na osnovu aplikacija - aplikacija se piše tako da je moguće napraviti izmene i sakriti ili učiniti dostupnim određene funkcionalnosti izmenom konfiguracionih fajlova. Ovo omogućava i tehniku tzv. dark launching - kada se sve funkcionalnosti instaliramo uz producioni softver i testiramo pod realnim opterećenjem, ali su te funkcionalnosti skrivene od krajnjih korisnika, dok se ne uverimo da je sve OK

55. Šta je plavo/zeleno obrazac lansiranja (blue/green pattern?)
Postoje dva produkciona okruženja (tzv. plavo i zeleno) u svakom momentu, ali samo jedno od njih prima saobraćaj od korisnika (za početak npr. plava). Nove verzije se instaliraju na pasivno okruženje (zeleno), tamo se sve testira, i kada smo uvereni da je sve ok, saobraćaj korisnika se preusmeri na to okruženje. • Ukoliko je potrebno izvršiti roll-back on se vrši jednostavno - saobraćaj se vrati na prvobitno aktivno okruženje (plavo). • Problem baza podataka: • dve baze - po jedna za svaku instancu koje se prilikom lansiranja nove verzije usinhronizuju kako bi novi sistem imao sve podatke sa do tada aktivnog. Problem se javlja ako se mora uraditi rolll-back, jer se moraju migrirati i sve transakcije koje su u međuvremenu nastale • odvajanje verzije baze od promena u aplikaciji - aplikacije se pišu tako da njihovo lansiranje ne zavisi od verzije baze: 1. dozvoljavaju se samo aditivne promene u bazi, 2. aplikacija ne pretpostavlja tačno određenu verziju baze da bi radila.

56. Šta je obrazac Canary release?
Automatizuje proces lansiranja nove verzije tako što se novi kod sukcesivno instalira na sve složenija okruženja, ali samo pod uslovom da je na prethodnom okruženju sve uspešno testirano. • Ukoliko se na određenom okruženju otkrije greška, radi se roll-back i novi kod se ne promoviše dalje. • Primer: • A1 - grupa servera za lokalnu upotrebu • A2 - grupa servera koju može da koristi limitirani broj krajnjih korisnika nakon što je potvrđen acceptance • A3 - puno produkciono okruženje

57. Šta je Cluster immune system pattern?
Modifikacija canary pattern-a tako što se povezuje monitroing sistem na produkciji sa automatizovanim release procesom, tako da kada monitoring sistem otkrije da performanse na produkcionom sistemu nakon pokretanja nove verzije padaju ispod određenih unapred

postavljenih parametara, automatski trigeruje roll-back. • Ovo dodatno smanjuje vreme reakcije i oporavka sistema

58. Koji su obrasci lansiranja koji se baziraju na samim aplikacijama? Zašto je potrebna automatizacija procesa isporuke?

Implementiranje mogućnosti uključivanja/isključivanja funkcionalanosti (feature toggles). • Obično se implementira tako što se delovi interfejsa vidljivi krajnjem korisniku i funkcionalnosti kojima se pristupa preko tog interfejsa “umotavaju” u uslovne naredbe • Omogućava lak povratak na verziju bez date funkcionalnosti (rolll-back), smanjenje funkcionalnosti uz očuvanje aktivnosti aplikacije (gracefull degradation of performance), povećanje otpornosti sistema baziranog na servisnim konceptima (moguće je implementirati funkcionalnost koja se oslanja na servis, čak i kada on ne radi u potpunosti i uključiti je kada sve proradi, ili isključiti ako servis ima problema)

• Korišćenje lansiranja aplikacije “u mraku” - dark launches • Ovaj konept je u suštini posledica mogućnosti da se funkcionalnosti uključe isključe. Ideja je da se aplikacija sa svim funkcionalnostima instalira na produkciono okruženje, ali da se određene funkcionalnosti ne uključuju za krajnjeg korisnika, ali se za to vreme testiraju u potpuno relanom okruženju. Krajnji korisnici su nesvesni da nova funkcionalnost postoji sve do momenta kada smo zadovoljni testiranjem i uključimo je i za njih

59. Koje korake procesa isporuke se može automatizovati?
Build, testiranje, deployment, konfiguracija, monitoring